

# Nesne Yönelimli Programlama

Hafta 2: Kontrol Yapıları ve Algoritmik Döngüler

Dr. Hüseyin Yüce

Marmara Üniversitesi Teknoloji Fakültesi

Mekatronik Mühendisliği Bölümü

# Hafta 2: Teorik ve Kavramsal Hedefler

Bu hafta hedeflenen temel kavramsal kazanımlar:

- Karar Yapıları ve Akış Denetimi:
- Tekil ve ikili karar mekanizmaları ( `if` , `if-else` )
- Çoklu alternatifler ve `else if` merdiveni
- Ayırık seçimler: `switch-case` ve `fall-through` dinamikleri
- Modern C++ ve İfadeler:
- Üçlü koşul operatörü ( `?` : Ternary)
- C++17 Başlatıcılı seçim yapıları ( `if` / `switch` with init)
- Algoritmik Döngü Mekanizmaları:
- Ön koşullu ( `while` ), son koşullu ( `do-while` ), sayaç kontrollü ( `for` )

# Hafta 2: Uygulamalı ve Algoritmik Hedefler

Laboratuvar oturumu ve problem çözme kazanımları:

- Mekatronik ve Robotik Akış Denetimi:
- Sensör eşik değer analizi ve güvenli duruş mantığı
- Konsol tabanlı sonlu durum makineleri (FSM) ve kullanıcı menüleri
- Temel Algoritmik Problemler:
- Asal sayı tespiti ve  $O(\sqrt{n})$  matematiksel optimizasyon
- Fibonacci dizisi ve terim kaydırma algoritması
- Faktöriyel hesaplama ve veri tipi taşması (*integer overflow*)
- Veri Analitiği ve Pratikler:
- Dinamik veri akışında en küçük, en büyük ve ortalama tespiti

# Motivasyon: Neden Kontrol Yapıları?

Yazılımlar gerçek dünyada doğrusal (*linear*) bir çizgide ilerleyemez:

- Karar Verme İhtiyacı: Programın dış dünya değişkenlerine (kullanıcı girdisi, sensör verisi, haberleşme paketi) dinamik tepki vermesi gerekir.
- Mekatronik Senaryosu: Otonom bir mobil robot hareket halindeyken:
  - Mesafe sensörü  $d < 15\text{ cm}$  algıladığında fren yapmalıdır.
  - Yol açıksa ilerlemeye devam etmelidir.
- Mantıksal Dallanma: Kontrol yapıları, program akışını koşullara göre farklı yürütme dallarına ayırır.

Özet: Kontrol yapıları, bir yazılımın dış dünyayı algılayıp tepki veren akıl ve karar mekanizmasıdır.

# Motivasyon: Neden Döngüler?

Tekrarlayan görevleri manuel kodlamak verimsiz ve hataya açıktır:

- DRY Prensipleri (Don't Repeat Yourself): Aynı işlemi 1000 kez yazmak yerine döngü kurgulayarak tek bir blok halinde ifade ederiz.
- Mekatronik Kontrol Döngüsü (Control Loop):
  - Gerçek zamanlı sistemlerde sensörler sürekli okunur.
  - PID kontrol algoritması hesaplanır.
  - Motor sürücüsüne PWM sinyali periyodik olarak iletilir.
- Veri Kümelerini İşleme: Diziler, listeler ve sensör tamponları döngülerle taranır.

**Sistem Döngüsü:** Bir robotun ana fonksiyonu, kapatma emri gelene kadar çalışan bir döngüden (`while (sistemAktif)`) ibarettir.

# Karşılaştırma Operatörleri

İki değeri kıyaslayıp `bool` ( `true` / `false` ) üretir:

| Operatör           | Anlamı              | Örnek                         |
|--------------------|---------------------|-------------------------------|
| <code>==</code>    | Eşit midir?         | <code>x == 10</code>          |
| <code>!=</code>    | Eşit değil midir?   | <code>x != 0</code>           |
| <code>&lt;</code>  | Küçük müdür?        | <code>hiz &lt; 50</code>      |
| <code>&lt;=</code> | Küçük veya eşit mi? | <code>hiz &lt;= 100</code>    |
| <code>&gt;</code>  | Büyük müdür?        | <code>sicaklik &gt; 85</code> |
| <code>&gt;=</code> | Büyük veya eşit mi? | <code>voltaj &gt;= 3.3</code> |

# Mantıksal Operatörler

Birden çok koşulu birleştirmek için kullanılır:

| Operatör | Anlamı                | Mantıksal Karşılık                    |
|----------|-----------------------|---------------------------------------|
| &&       | Mantıksal VE (AND)    | İki koşul da <code>true</code> olmalı |
|          | Mantıksal VEYA (OR)   | En az biri <code>true</code> olmalı   |
| !        | Mantıksal DEĞİL (NOT) | Değeri tersine çevirir                |

```
1 // Örnek Birleşim:
2 if (sicaklik > 80 && fanCalisiyorMu == false) {
3     acilSogutmaBaslat();
4 }
```

# Kısa Devre Değerlendirmesi (Short-Circuit)

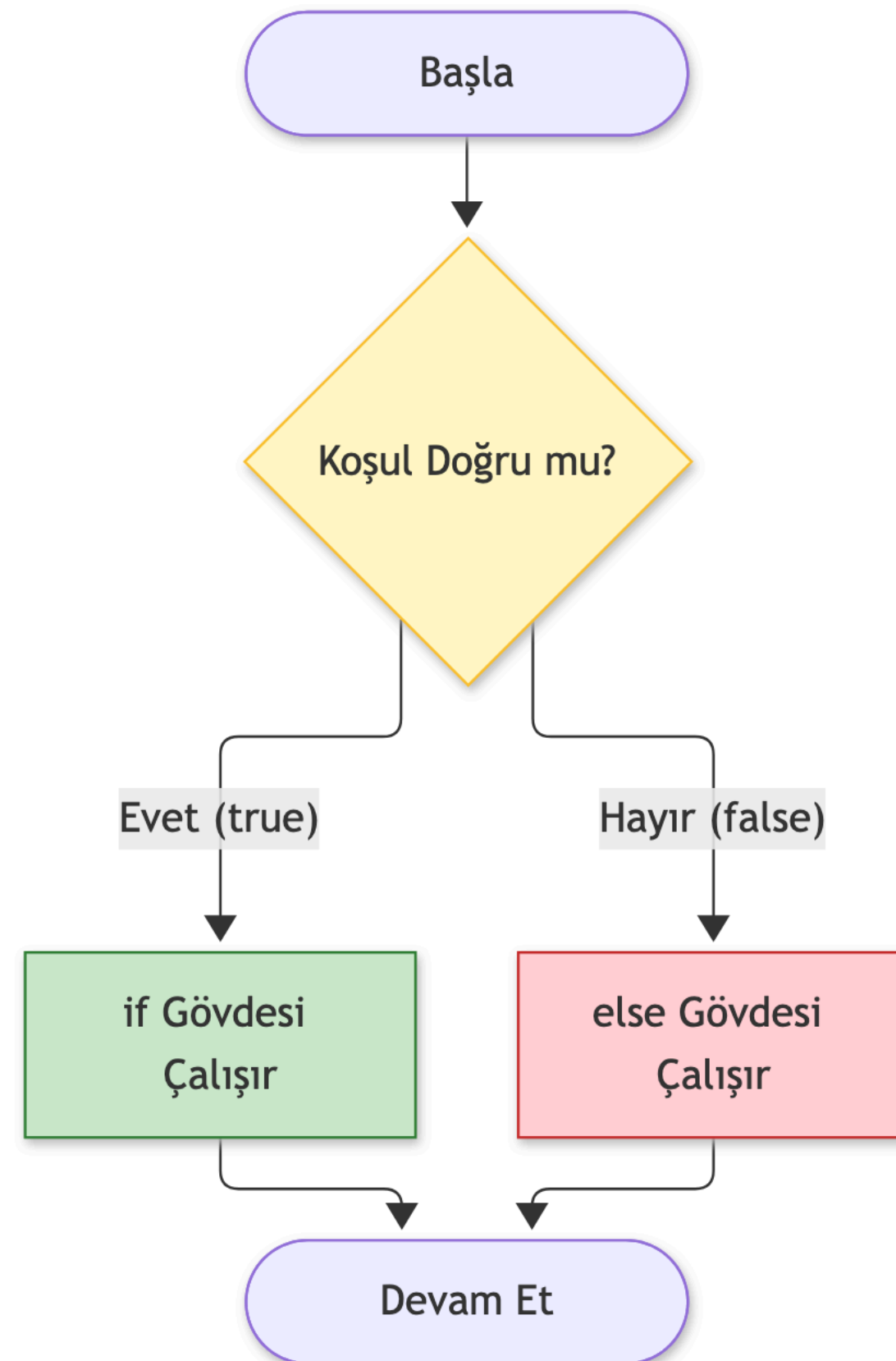
C++ mantıksal ifadeleri soldan sağa değerlendirir ve sonucu kesinleştiği anda durur:

- Mantıksal VE ( `&&` ): İlk koşul `false` ise, ikinci koşul asla kontrol edilmez ( `false && ...` her zaman `false` 'tur).
- Mantıksal VEYA ( `||` ): İlk koşul `true` ise, ikinci koşul asla kontrol edilmez ( `true || ...` her zaman `true` 'dur).
- Mühendislik Avantajı (Güvenli Kontrol): Sıfıra bölme ve geçersiz işaretçi hatalarını önler:

```
1 // payda 0 ise sağ taraf (100 / payda) asla çalıştırılmaz, program çökmez!  
2 if (payda != 0 && (pay / payda) > 5) {  
3     cout << "Oran geçerli sınırdadır." << endl;  
4 }
```

# if-else Karar Yapısı

Bir koşulun doğruluğuna göre iki yoldan birini seçer:



# Sözdizimi ve Kurallar

```
1  if (koşul) {  
2      // Koşul true ise çalışır  
3  } else {  
4      // Koşul false ise çalışır  
5  }
```

- `koşul` ifadesi parantez `(...)` içine yazılmalıdır.
- `else` bloğu zorunlu değildir; tek başına `if` de kullanılabilir.
- Gövdede tek bir satır dahi olsa süslü parantez `{ }` kullanımı en iyi uygulamadır (*Best Practice*).

# if-else Mekanik Örneği

Engel mesafesi denetimi ve otonom tepki:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      double engelMesafesi = 12.5; // cm
6
7      if (engelMesafesi < 15.0) {
8          cout << "[UYARI] Engel yakın! Fren yapılıyor."
9              << endl;
10     } else {
11         cout << "[BILGI] Yol açık. İlerlemeye devam."
12             << endl;
13     }
14     return 0;
15 }
```

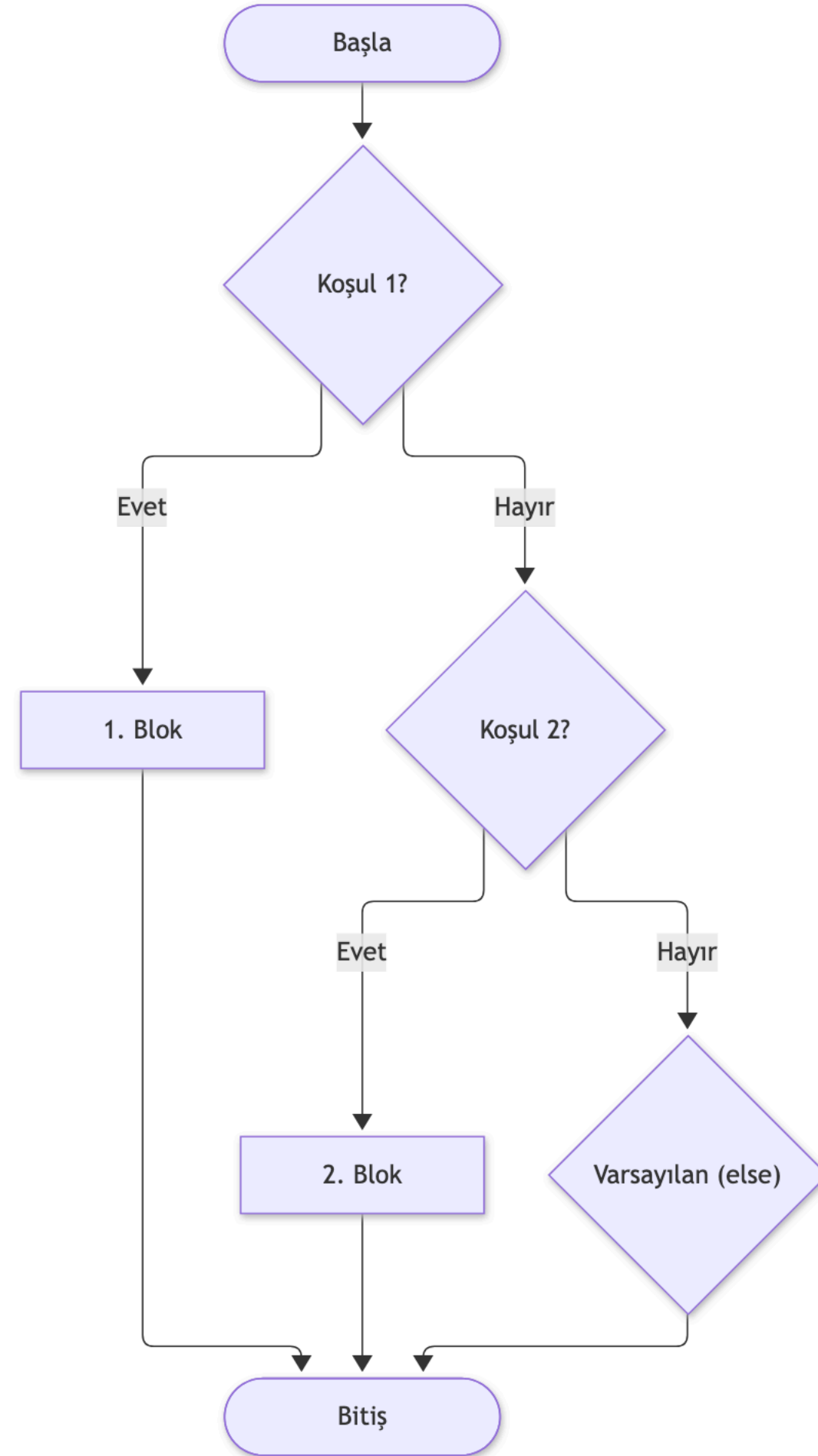
# Kod İncelemesi

- Satır 7: `engelMesafesi < 15.0` ilişkisel ifadesi değerlendirilir.
- Değer `12.5` olduğu için koşul `true` üretir.
- Satır 8-9: Yalnızca `if` bloğu içerisindeki kodlar çalıştırılır.
- `else` bloğu tamamen atlanır ve program `return 0;` satırına geçer.
- Ekran Çıktısı:

```
1  [UYARI] Engel yakın! Fren yapılıyor.
```

# else if Merdiveni

İkiden fazla alternatif koşulun sıralı denetimi:



# Değerlendirme Mantığı

- Koşullar yukarıdan aşağıya doğru sırayla test edilir.
- İlk sağlanan ( `true` olan) bloğun gövdesi yürütülür.
- O blok bittiğinde diğer tüm `else if` ve `else` adımları atlanır!
- Hiçbir koşul sağlanmazsa sondaki `else` bloğu yürütülür.
- Sıralama mantığı kritiktir; genel koşullar öne yazılırsa alttaki özel koşullar ölü koda (*dead code*) dönüşür.

# else if Kod Örneği

Motor devir aralığına göre çalışma modu belirleme:

```
1  int devir = 2400; // RPM
2
3  if (devir == 0) {
4      cout << "Motor Durduruldu." << endl;
5  } else if (devir < 1000) {
6      cout << "Düşük Hız Modu (Eko)" << endl;
7  } else if (devir <= 3000) {
8      cout << "Nominal Çalışma Modu" << endl;
9  } else {
10     cout << "Aşırı Hız Uyarısı!" << endl;
11 }
```

# Tasarım Notları

- `devir = 2400` değeri için:
  - `devir == 0` → `false`
  - `devir < 1000` → `false`
  - `devir <= 3000` → `true`
- Program `"Nominal Çalışma Modu"` yazdırır ve alttaki `else` bloğuna hiç bakmadan çıkar.
- Aralıklar birbirini dışlayan (*mutually exclusive*) mantıkla kurgulanmalıdır.

# Ternary (Üçlü Koşul) Operatörü ( ? : )

Basit `if-else` kararlarını tek satırda ifade eder:

```
1 // Standart if-else:
2 int limit = 100;
3 int sonuc;
4 if (limit > 50) {
5     sonuc = 1;
6 } else {
7     sonuc = 0;
8 }
9
10 // Ternary karşılığı:
11 int sonuc = (limit > 50) ? 1 : 0;
```

# Kullanım Alanı ve Kurallar

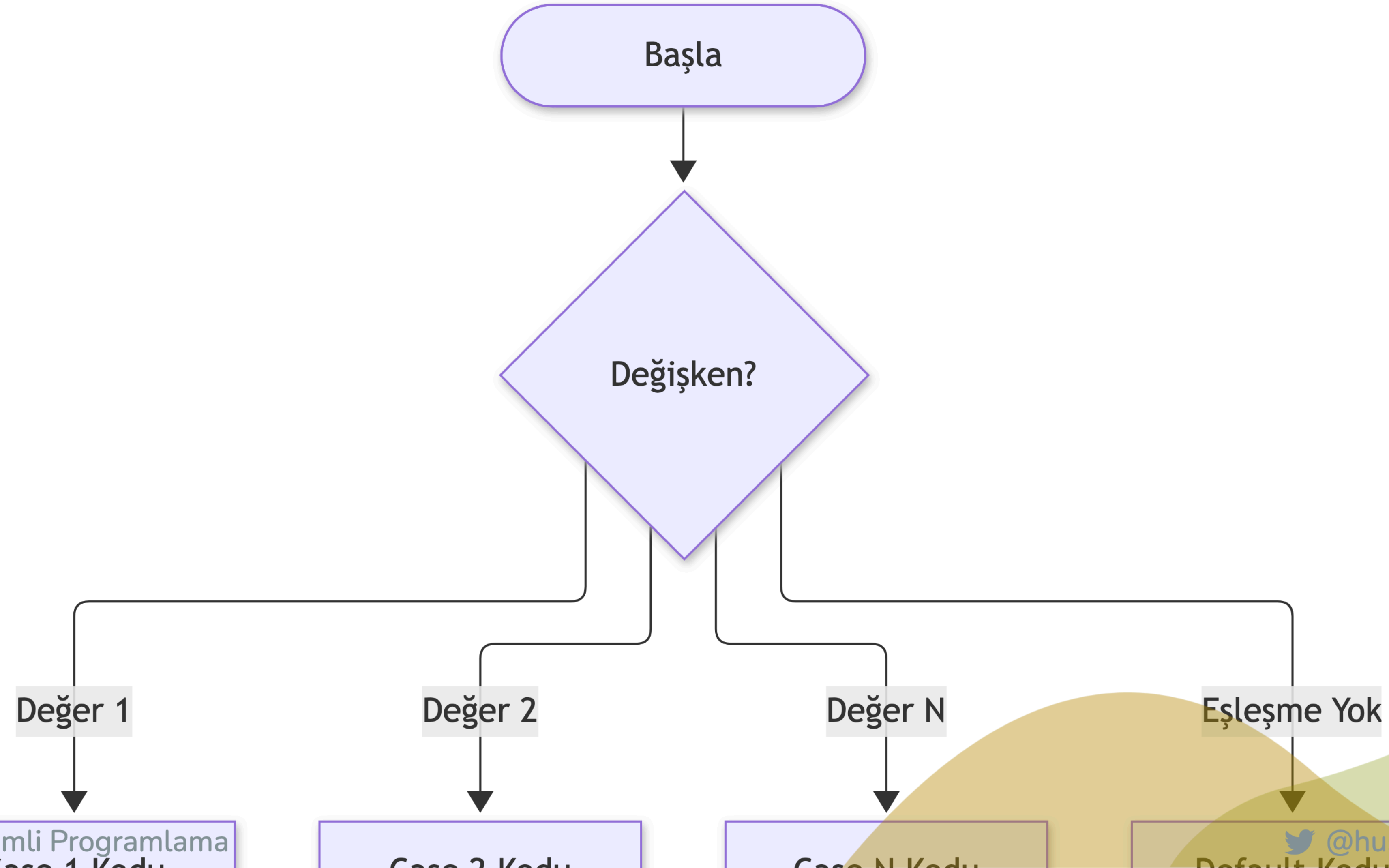
- Sözdizimi: `sonuc = (koşul) ? deger_dogru : deger_yanlis;`
- Değer atamalarında, `return` ifadelerinde veya `cout` akışlarında çok pratiktir.
- Örnek:

```
1  cout << "Durum: "  
2      << (aktif ? "AÇIK" : "KAPALI")  
3      << endl;
```

- Karmaşık, iç içe (*nested*) ternary kullanımı kodun okunabilirliğini bozar; kaçınılmalıdır!

# switch-case Yapısı

Tek bir değişkenin ayırık sabit değerlerle eşleşmesini denetler:



# Sözdizimi Şablonu

```
1  switch (ifade) {  
2      case SABIT_1:  
3          // Komutlar  
4          break;  
5      case SABIT_2:  
6          // Komutlar  
7          break;  
8      default:  
9          // Hiçbiri eşleşmezse  
10         break;  
11 }
```

- `if-else` merdivenine göre ayırık durumlarda daha temiz ve optimize çalışır.

# switch-case Kritik Kuralları ve Kısıtlamaları

switch-case kullanırken derleyicinin şart koştuğu katı kurallar:

- Yalnızca Tamsayı ve Karakter Tipleri:
- switch (ifade) içerisindeki ifade yalnızca tamsayı ( int , short , long ), karakter ( char ) veya enum olabilir.
- float , double veya std::string asla kullanılamaz! (Derleme hatası verir).
- Sabit Case Değerleri:
- case ifadesinin yanındaki değer derleme zamanı sabiti ( *compile-time constant* ) olmalıdır ( case 5: , case 'A': ). Değişken yazılamaz ( case x: geçersizdir).
- break Anahtar Kelimesi:
- Eşleşen blok tamamlandığında switch bloğundan çıkışı sağlar. Unutulursa akış alt satırdaki case'e düşer ( *Fall-Through* ).

# switch-case Robot Menü Örneği

Robot kontrol terminali komut yorumlayıcı:

```
1  char komut = 'W';
2
3  switch (komut) {
4      case 'W':
5      case 'w':
6          cout << "İleri Hareket" << endl;
7          break;
8      case 'S':
9      case 's':
10         cout << "Geri Hareket" << endl;
11         break;
12     case 'X':
13     case 'x':
14         cout << "Acil Duruş!" << endl;
15         break;
16     default:
17         cout << "Bilinmeyen Komut!" << endl;
18         break;
19 }
```

# Tasarım Notları

- Çoklu Case Etiketi: Hem büyük harf `'W'` hem küçük harf `'w'` aynı bloğu tetikler.
- `default` Bloğu: Kullanıcı tanımlı olmayan bir karakter girdiğinde savunmacı programlama (*defensive programming*) gereği hata mesajı üretir.
- Her durumun sonunda `break;` ile `switch` bloğundan çıkış garanti altına alınmıştır.

# switch-case Tehlikesi: Fall-Through

`break` konulmadığında akışın alttaki bloklara sızması:

```
1  int vites = 1;
2
3  switch (vites) {
4      case 1:
5          cout << "1. Vites ";
6      case 2:
7          cout << "2. Vites ";
8      case 3:
9          cout << "3. Vites ";
10         break;
11     default:
12         cout << "Boşta";
13 }
```

Ekran Çıktısı:

```
1  1. Vites 2. Vites 3. Vites
```

# Fall-Through Neden Oluşur?

- C++ `switch` mekanizmasında etiketler ( `case` ) sadece birer atlama noktasıdır (*jump label*).
- `case 1` eşleştğinde `break` görene kadar aşağıdaki tüm kodları koşula bakılmaksızın yürütür!
- Çoğu zaman bu durum programcının unuttuğu bir mantık hatasıdır (*bug*).
- Ancak bazen bilinçli olarak ardışık durumları işletmek için de kullanılır.

# Modern C++17: `[[fallthrough]]` Özelliği

Bilinçli yapılan geçişleri derleyiciye bildirme standardı:

- Modern derleyiciler ( `-Wall -Wextra` ), `break` konulmamış case blokları için uyarı verir.
- Eğer kasıtlı olarak bir alt case'e geçmek istiyorsak C++17 ile gelen `[[fallthrough]]`; özniteliği kullanılır:

```
1  switch (cihazDurumu) {  
2      case HATA_KRITIK:  
3          sesliAlarmCal();  
4          [[fallthrough]]; // Derleyiciye bilinçli geçiş yaptığımızı belirtir (uyarıyı sustu)  
5      case HATA_UYARI:  
6          kayitDefterineYaz();  
7          break;  
8      case NORMAL:  
9          islemeDevamEt();  
10         break;  
11 }
```

# Modern C++17: Başlatıcılı `if` Yapısı

Değişken tanımını doğrudan `if` ifadesi içine alma (*if with initializer*):

```
1 // C++14 ve Öncesi:
2 int hataKodu = sensorOku();
3 if (hataKodu != 0) {
4     hataYonet(hataKodu);
5 }
6 // hataKodu değişkeni burada hala yaşıyor!
7
8 // C++17 Standart Yaklaşımı:
9 if (int hataKodu = sensorOku(); hataKodu != 0) {
10     hataYonet(hataKodu);
11 }
12 // hataKodu if bloğu dışında yok edilir!
```

# Avantajları ve Kapsam Denetimi

- Değişken Kirliliğini Önler: `hataKodu` değişkeni yalnızca `if` ve `else` blokları içinde geçerlidir (*scope pollution* engellenir).
- Güvenlik: Aynı isimli değişkenlerin fonksiyon genelinde çakışmasını önler.
- `switch` ile Kullanım:

```
1  switch (auto durum = getStatus(); durum) {  
2      case OK: break;  
3      case FAIL: break;  
4  }
```

# Karar Yapılarında Sık Yapılan Hatalar

Sınavlarda ve projelerde en çok karşılaşılan tuzaklar:

- Atama ( `=` ) ile Karşılaştırma ( `==` ) Karıştırma:

```
1  if (sayi = 0) { ... } // HATA: sayi'ya 0 atanır ve ifade false üretir! Blok asla çalışmaz.  
2  if (sayi == 0) { ... } // DOĞRU
```

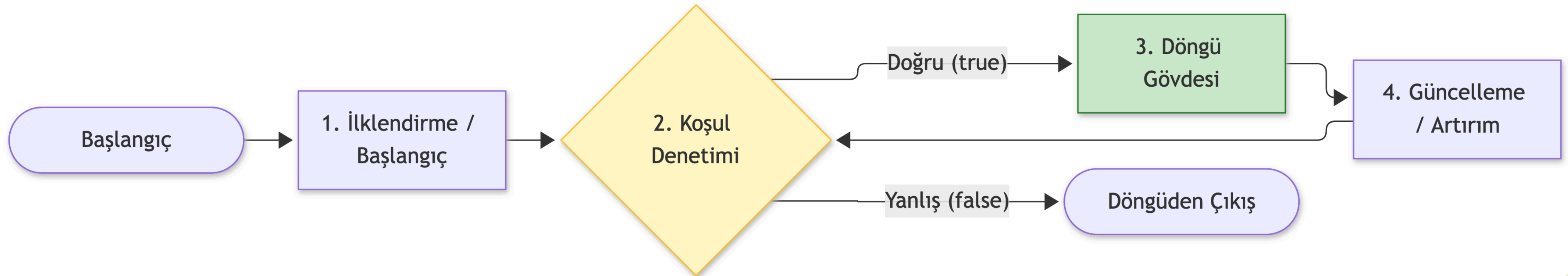
- Koşul Parantezinden Sonra Yanlışlıkla Noktalı Virgül Koyma:

```
1  if (x > 10); // DİKKAT: Boş ifade sonlanır!  
2  {  
3      x = 0;    // Koşula bağlı olmaksızın HER ZAMAN çalışır!  
4  }
```

- Kayan Noktalı Sayılarda Eşitlik ( `==` ) Aramak: `double` tiplerinde hassasiyet farkı nedeniyle `if (d == 1.0)` yerine toleranslı kıyaslama ( `std::abs(d - 1.0) < 1e-6` ) yapılmalıdır.

# Döngülere Giriş (Tekrarlı İşlem Mantığı)

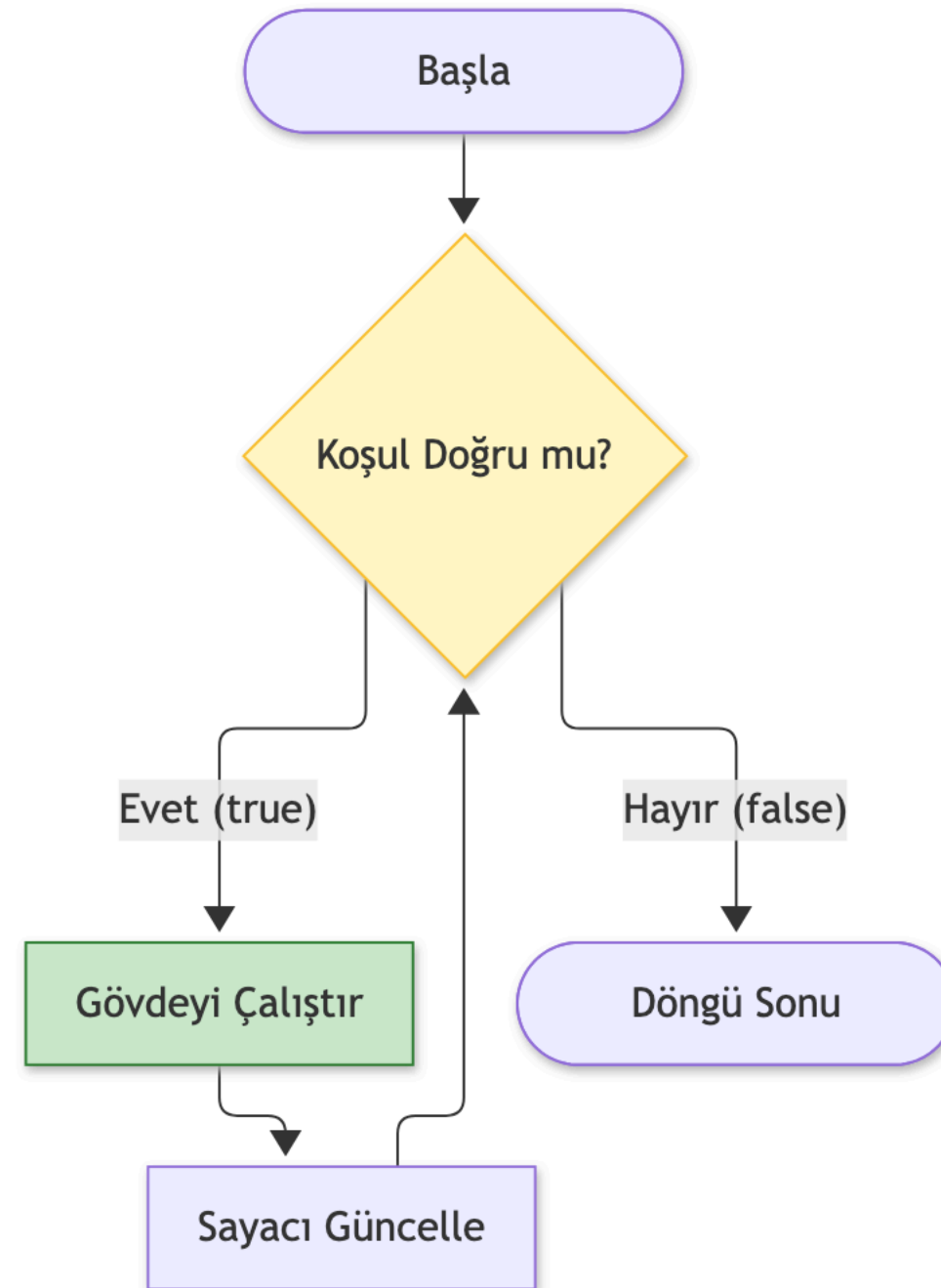
Bir algoritmanın belirli kurallar çerçevesinde tekrar tekrar işletilmesidir:



- Bir döngünün sağlıklı çalışması için 3 temel unsur eksiksiz olmalıdır:
  1. Başlangıç Durumu: Sayacın veya durum değişkeninin ilk değeri.
  2. Döngü Koşulu: Döngünün ne zaman devam edeceğini belirleyen mantıksal ifade.
  3. Güncelleme Adımı: Her tur sonunda sayacı hedefe yaklaştıran adım.

# while Döngüsü (Ön Koşullu Döngü)

Koşul doğru olduğu sürece çalışır:



# Sözdizimi ve Mantık

```
1 while (koşul) {  
2     // Koşul true olduğu sürece çalışır  
3     // Güncelleme ifadesi burada olmalı  
4 }
```

- Ön Koşul Mantığı: Koşul en başta denetlenir.
- Eğer koşul ilk başta `false` ise, döngü gövdesi hiç çalıştırılmaz (0 kez).
- Kaç tur döneceği önceden bilinmeyen, harici bir olaya veya girdiye bağlı durumlarda idealdir.

# while Döngüsü C++ Örneği

Basit sayaç ve toplam hesabı:

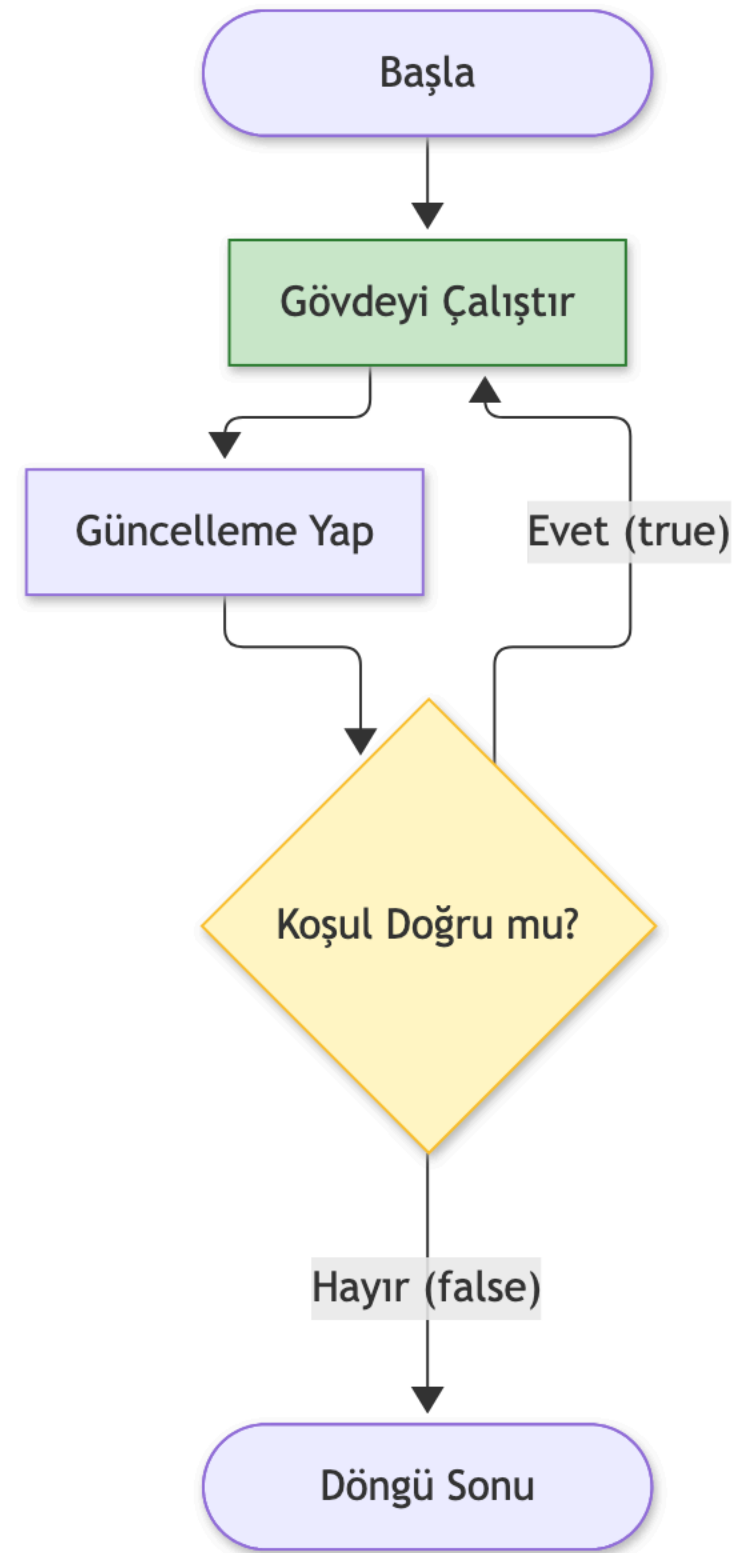
```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int sayac = 1;
6      int toplam = 0;
7
8      while (sayac <= 5) {
9          toplam += sayac;
10         cout << "Adım: " << sayac
11             << ", Toplam: " << toplam << endl;
12         sayac++; // Güncelleme adımı
13     }
14     return 0;
15 }
```

# Çalışma Adımları

- `sayac = 1` iken başlar.
- Her iterasyonda `toplam` güncellenir ve `sayac++` ile bir artırılır.
- `sayac = 6` olduğunda `sayac <= 5` koşulu `false` olur ve döngü biter.
- Kritik Uyarı: `sayac++` satırı unutulursa `sayac` sonsuza kadar 1 kalır ve sonsuz döngü (infinite loop) oluşur!

# do-while Döngüsü (Son Koşullu Döngü)

Gövdesi en az bir kez mutlaka çalışan döngü:



# Sözdizimi ve Kurallar

```
1  do {  
2      // işlemler yürütülür  
3  } while (koşul); // DİKKAT: Sondaki ';' zorunludur!
```

- Son Koşul Mantığı: Koşul kontrolü gövde çalıştıktan sonra yapılır.
- Koşul ilk başta `false` bile olsa, gövde en az 1 kez mutlaka çalışır.
- Kullanıcıdan girdi alma, menü gösterme ve donanım ilk değer atama işlemlerinde tercih edilir.

# do-while ile Güvenli Girdi Alma

Kullanıcı geçerli bir seçim yapana kadar sorma:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int secim;
6      do {
7          cout << "\n--- KONTROL PANELİ ---" << endl;
8          cout << "1. Sensörleri Oku" << endl;
9          cout << "2. Motorları Başlat" << endl;
10         cout << "0. Çıkış" << endl;
11         cout << "Seçiminiz (0-2): ";
12         cin >> secim;
13     } while (secim < 0 || secim > 2);
14
15     cout << "Geçerli seçim yapıldı: "
16          << secim << endl;
17     return 0;
18 }
```

# Tasarım Notları

- Menünün kullanıcıya en az bir kez gösterilmesi şarttır; bu nedenle `do-while` mükemmel bir çözümdür.
- Eğer kullanıcı 5 girerse:
  - `secim < 0 || secim > 2` koşulu `true` olur.
  - Döngü başa sarar ve menüyü tekrar ekrana basar.
- Kullanıcı 0, 1 veya 2 girdiğinde döngü sonlanır.

# while ve do-while Karşılaştırması

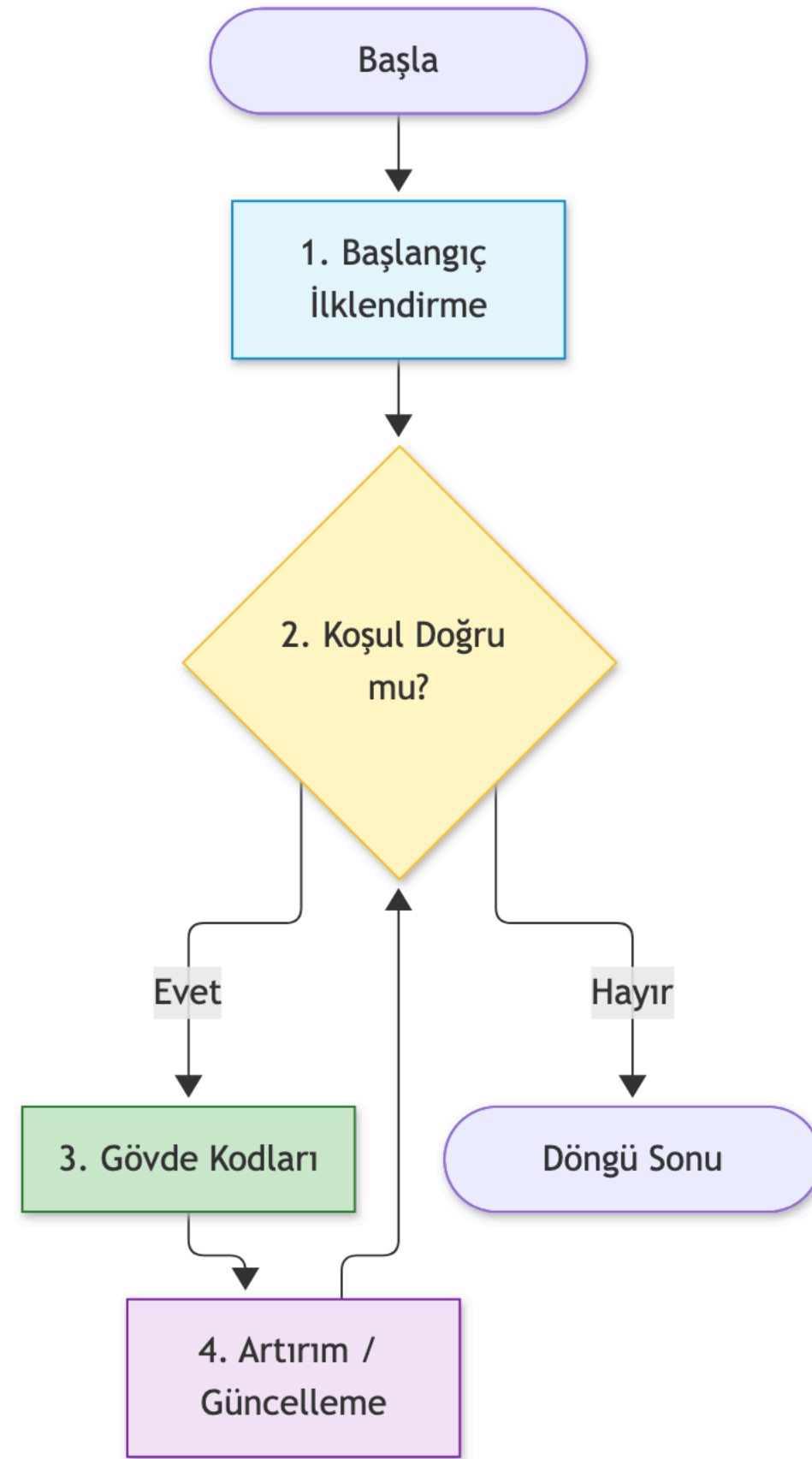
İki döngü türü arasındaki temel mühendislik farkları:

| Özellik                | while Döngüsü                         | do-while Döngüsü                                     |
|------------------------|---------------------------------------|------------------------------------------------------|
| Koşul Denetim Yeri     | Döngünün başında (Girişte)            | Döngünün sonunda (Çıkışta)                           |
| Minimum Çalışma Sayısı | 0 kez (Koşul baştan false ise)        | 1 kez (Gövde koşuldan önce çalışır)                  |
| Sözdizimi Bitişi       | <code>while (koşul) { ... }</code>    | <code>do { ... } while (koşul);</code> (Sonda ; var) |
| İdeal Senaryolar       | Veri kuyruğu okuma, sınır kontrolleri | Menü sistemleri, kullanıcı girdisi doğrulama         |

**Sınav İpucu:** `do-while` yapısının sonundaki noktalı virgül ( `;` ) unutulursa derleyici sentaks hatası verir. `while` yapısında ise gövde açılışından önce noktalı virgül konulmamalıdır!

# for Döngüsü (Sayaç Kontrollü)

İterasyon sayısı önceden belli olan durumlar için en derli toplu yapı:



# for Döngüsünün Anatomisi

```
1  for (başlangıç; koşul; güncelleme) {  
2      // Gövde  
3  }
```

- 1. Başlangıç: Döngü sayacı tanımlanır ve ilk değer verilir ( `int i = 0` ). Yalnızca 1 kez çalışır.
- 2. Koşul: Her tur öncesi denetlenir ( `i < 10` ).
- 3. Gövde: Koşul `true` ise yürütülür.
- 4. Güncelleme: Gövde bitiminde sayacı değiştirir ( `i++` ).

# for Döngüsü C++ Uygulaması

Çift sayıların toplamını hesaplama:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int ciftToplam = 0;
6
7      for (int i = 0; i <= 10; i += 2) {
8          ciftToplam += i;
9          cout << "i: " << i
10             << ", Ara Toplam: " << ciftToplam << endl;
11     }
12
13     cout << "Sonuç: " << ciftToplam << endl;
14     return 0;
15 }
```

# Kod İncelemesi

- `int i = 0` : Sayaç 0'dan başlar ve kapsamı (*scope*) sadece `for` bloğu ile sınırlıdır.
- `i += 2` : Sayaç ikişer ikişer artırılır (0, 2, 4, 6, 8, 10).
- `i = 12` olduğunda `i <= 10` koşulu bozulur ve döngü biter.
- Çıktı:

```
1  Sonuç: 30
```

# Modern C++: Aralık Tabanlı `for` (Range-Based)

C++11 ile gelen modern koleksiyon tarama yöntemi:

```
1 // C++98 / Klasik İndeksli Yaklaşım:
2 int dizi[5] = {10, 20, 30, 40, 50};
3 for (int i = 0; i < 5; i++) {
4     cout << dizi[i] << " ";
5 }
6
7 // C++11 Range-Based for Yaklaşımı:
8 for (int eleman : dizi) {
9     cout << eleman << " ";
10 }
```

# Avantajları ve Referans Kullanımı

- Sınır Güvenliği: İndeks taşması (*out of bounds*) riski tamamen ortadan kalkar.
- Temiz Kod: Sayaç tanımlama ve artırma detaylarıyla uğraşılmaz.
- Performans (Referans ile Geçiş): Büyük nesnelerde kopyalamayı önlemek için `const auto&` kullanılır:

```
1  for (const auto& deger : buyukListe) {  
2      cout << deger << endl;  
3  }
```

# İç İçe Döngüler (Nested Loops)

Bir döngünün gövdesinde başka bir döngünün yer alması:

- Çalışma Prensipleri: Dış döngünün her 1 adımı için, içteki döngü baştan sona tüm turlarını tamamlar.
- Kullanım Alanları: İki boyutlu matrisler, robotik piksel haritaları, koordinat taramaları.
- Karmaşıklık Uyarısı: Dış döngü  $N$ , iç döngü  $M$  kez dönüyorsa toplam işlem sayısı  $N \times M$  olur ( $O(N^2)$ ).

```
1 // 3 Satır, 4 Sütunluk Koordinat Izgarası:
2 for (int satir = 1; satir <= 3; satir++) {
3     for (int sutun = 1; sutun <= 4; sutun++) {
4         cout << "(" << satir << "," << sutun << ") ";
5     }
6     cout << endl; // Her satır bitiminde alt satıra geç
7 }
```

# Akış Denetimi: `break` İfadesi

Döngüyü koşulun bitmesini beklemeden anında sonlandırır:

```
1  int hedef = 7;
2
3  for (int i = 1; i <= 10; i++) {
4      if (i == hedef) {
5          cout << "Hedef " << i
6              << " bulundu, döngü kesiliyor!"
7              << endl;
8          break; // Döngüyü burada anında bitirir
9      }
10     cout << "Aranıyor: " << i << endl;
11 }
```

# Mekatronik Benzetimi: Acil Durdurma

- `break` , döngünün gövdesinden derhal dışarı atlar.
- Arama algoritmalarında aranan veri bulunduktan sonra gereksiz turları önleyerek CPU tasarrufu sağlar.
- Mekatronikte acil durum stop (*E-Stop*) sinyali geldiğinde ana kontrol döngüsünden çıkmak için kullanılır.
- Not: Yalnızca içinde bulunduğu en içteki döngüyü sonlandırır.

# Akış Denetimi: `continue` İfadesi

Mevcut turun geri kalanını atlayıp bir sonraki tura geçer:

```
1 // Sadece tek sayıları ekrana yazdır:
2 for (int i = 1; i <= 6; i++) {
3     if (i % 2 == 0) {
4         // Çift sayı ise alt satırları atla!
5         continue;
6     }
7     cout << "Tek Sayı: " << i << endl;
8 }
```

Çıktı:

```
1 Tek Sayı: 1
2 Tek Sayı: 3
3 Tek Sayı: 5
```

# `continue` Çalışma Mantığı

- Döngüyü tamamen kapatmaz; sadece o anki iterasyonu yarıda keser.
- `for` Döngüsünde: Doğrudan güncelleme adımına ( `i++` ) atlar, ardından koşulu kontrol eder.
- `while` Döngüsünde: Doğrudan koşul kontrolüne atlar (Dikkat: sayaç artırma satırı `continue` 'nun altında kalırsa sonsuz döngü oluşur!).
- Hatalı veya gürültülü sensör verilerini filtreleyip atlamak için idealdir.

# Döngülerde Sık Yapılan Hatalar ve Sınav Tuzakları

Öğrencilerin döngü kurgularken düştüğü tipik mantık hataları:

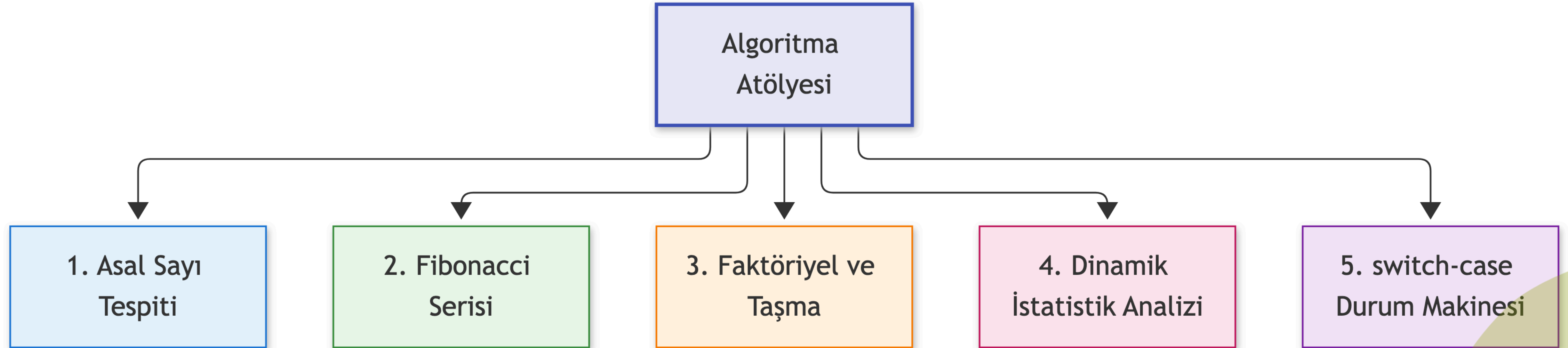
- Döngü Başlığından Sonra `;` Koymak:

```
1  int i = 0;  
2  while (i < 10); // HATA: Boş döngü sonsuza kadar döner, program kilitlenir!  
3  { i++; }
```

- Güncelleme Adımını Unutmak (Sonsuz Döngü): `while` veya `do-while` gövdesinde sayaç artırılmazsa koşul hep `true` kalır.
- Bir Eksik / Bir Fazla Hatası (Off-by-One Error): `<` yerine `<=` kullanmak sonucunda dizinin sınırlarını aşmak (*Buffer overflow*).
- Kayan Noktalı Sayı ile Eşitlik Döngüsü: `for (double d = 0.0; d != 1.0; d += 0.1)` → Küsürat yuvarlama hatası nedeniyle `d` asla tam 1.0 olmayabilir!

# Laboratuvar ve Algoritma Atölyesi

Temel algoritmik yapıların adım adım analizi ve C++ uygulamaları:



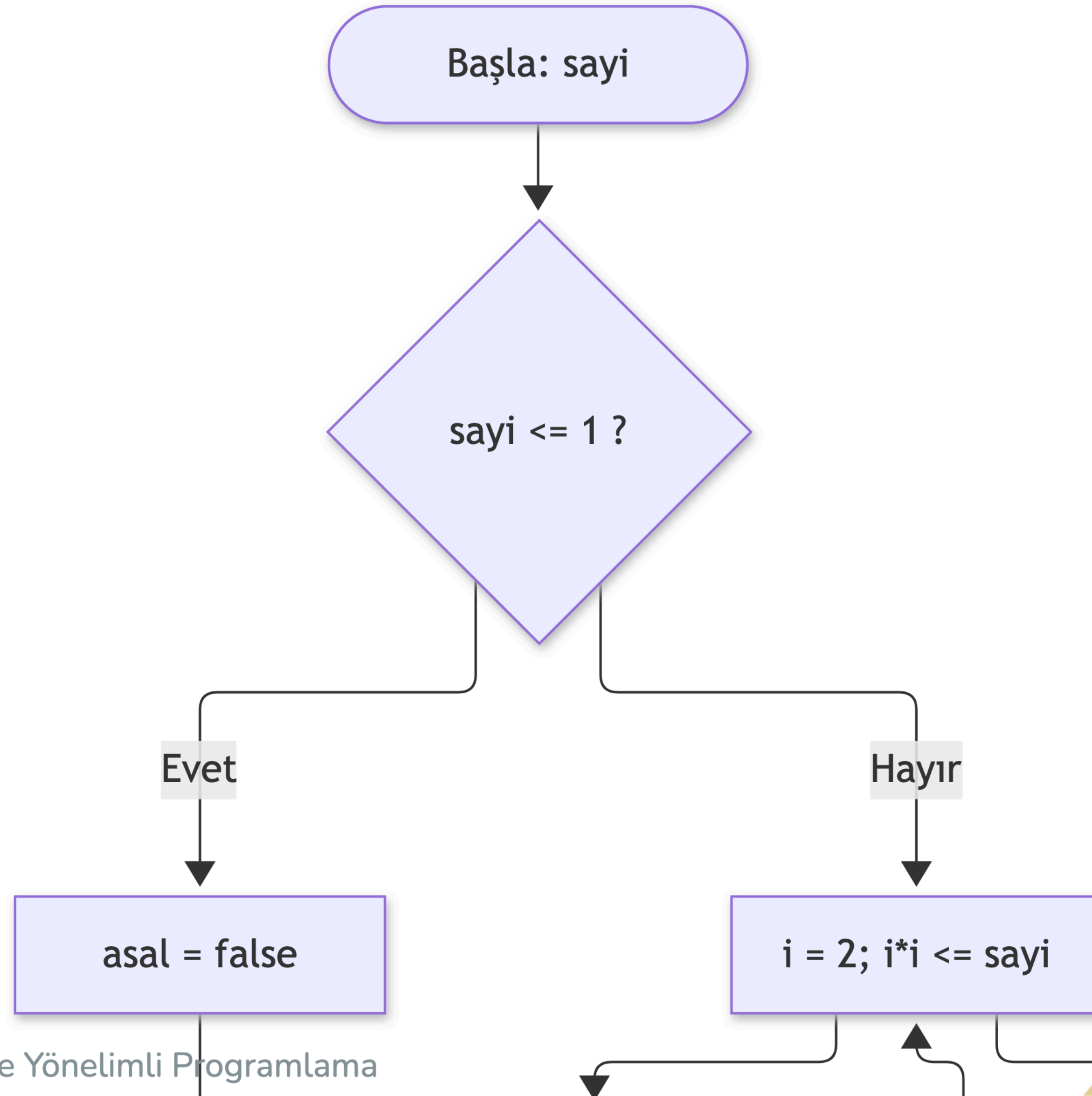
Her problemde: Problem Tanımı → Matematiksel Model → Akış Şeması → C++ Kodu sırası izlenecektir.

# Alıştırma 1: Asal Sayı Tespiti

Problem: Kullanıcıdan alınan  $N$  tamsayısının asal olup olmadığını belirleyin.

- Tanım: 1 ve kendisi hariç pozitif böleni olmayan, 1'den büyük doğal sayılardır.
- Matematiksel Optimizasyon:
  - Sayıyı 2'den  $N - 1$ 'e kadar denemek  $O(N)$  sürer.
  - Oysa bir sayının böleni varsa en az biri  $\leq \sqrt{N}$  olmak zorundadır!
  - Döngüyü  $i \times i \leq N$  şartıyla sınırlamak karmaşıklığı  $O(\sqrt{N})$  seviyesine düşürür.

# Algoritma Akış Şeması



# Alıştırma 1: Asal Sayı C++ Çözümü

Modern C++ ve optimize edilmiş bölen kontrolü:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int sayi;
6      cout << "Pozitif bir tamsayı girin: ";
7      cin >> sayi;
8
9      bool asalMi = true;
10
11     if (sayi <= 1) {
12         asalMi = false; // 1 ve daha küçük sayılar asal değildir
13     } else {
14         // Matematiksel optimizasyon: i <= sqrt(sayi) yerine i * i <= sayi
15         for (int i = 2; i * i <= sayi; i++) {
16             if (sayi % i == 0) {
17                 asalMi = false; // Tam bölen bulundu, asal olamaz
18                 break;          // Diğer sayılara bakmaya gerek yok!
19             }
20         }
21     }
```

# Alıştırma 2: Fibonacci Serisi

Problem: İlk  $N$  terimlik Fibonacci serisini ekrana yazdırın.

- Dizi Tanımı: Her terim, kendisinden önceki iki terimin toplamıdır:

$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$

- Yineleme Bağıntısı (Recurrence):

- $F(0) = 0$

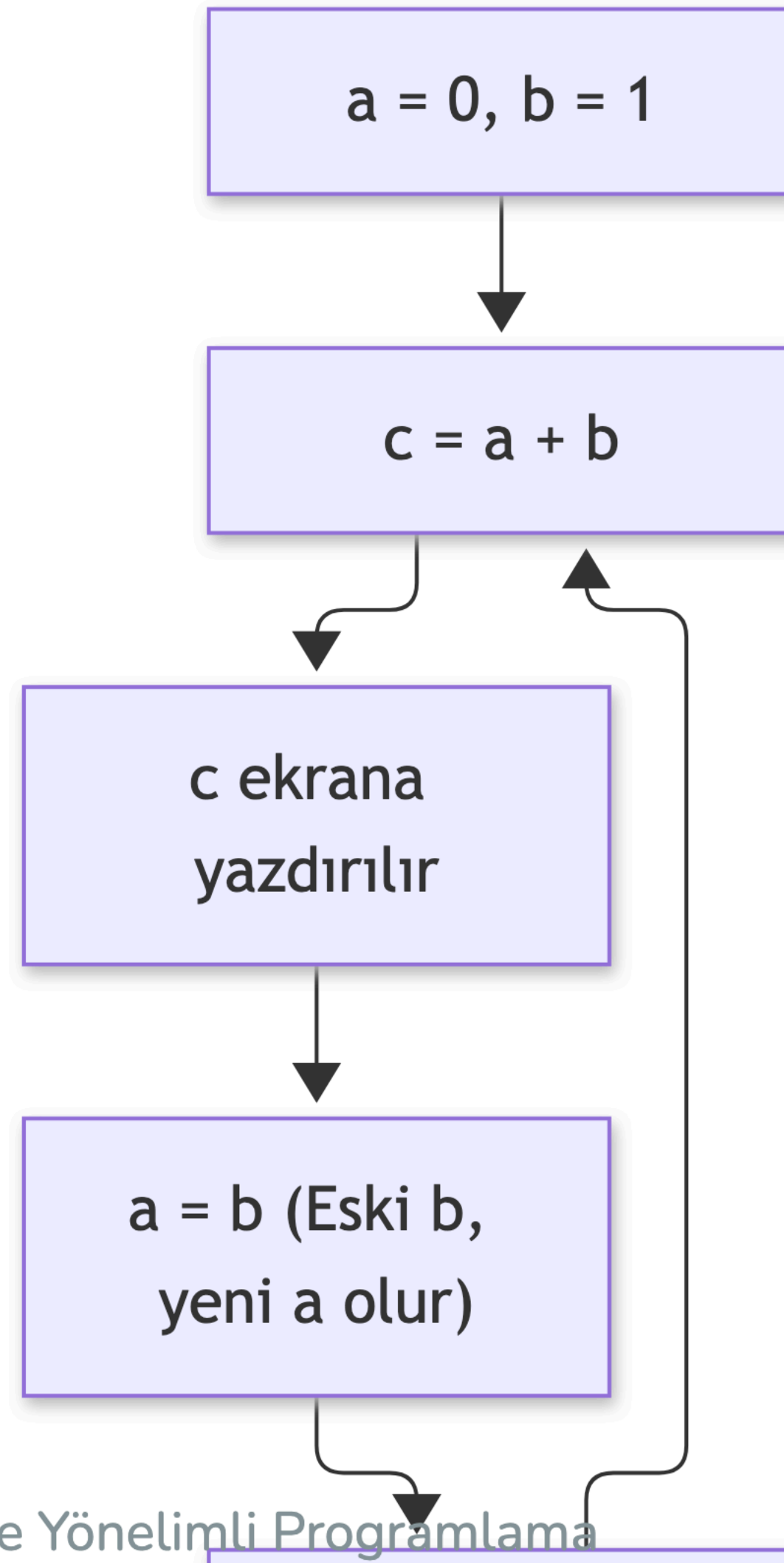
- $F(1) = 1$

- $F(n) = F(n - 1) + F(n - 2) \quad (n \geq 2)$

- Bellek Optimizasyonu: Tüm diziyi saklamak yerine sadece son 2 terimi ( `a` , `b` ) tutmak  $O(1)$  bellek harcar.

# Değişken Kaydırma Mantığı

Her adımda yeni terim üretilir ve değişkenler kaydırılır:



# Alıştırma 2: Fibonacci C++ Çözümü

İlk  $N$  terimi ekrana basan C++ programı:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int terimSayisi;
6      cout << "Üretilecek Fibonacci terim sayısını girin (N >= 2): ";
7      cin >> terimSayisi;
8
9      if (terimSayisi < 1) {
10         cout << "En az 1 terim istenmelidir!" << endl;
11         return 1;
12     }
13
14     long long a = 0; // F(0)
15     long long b = 1; // F(1)
16
17     cout << "Fibonacci Serisi: ";
18     if (terimSayisi == 1) {
19         cout << a << endl;
20     }
21     return 0;
22 }
```

# Alıştırma 3: Faktöriyel ve Veri Taşması

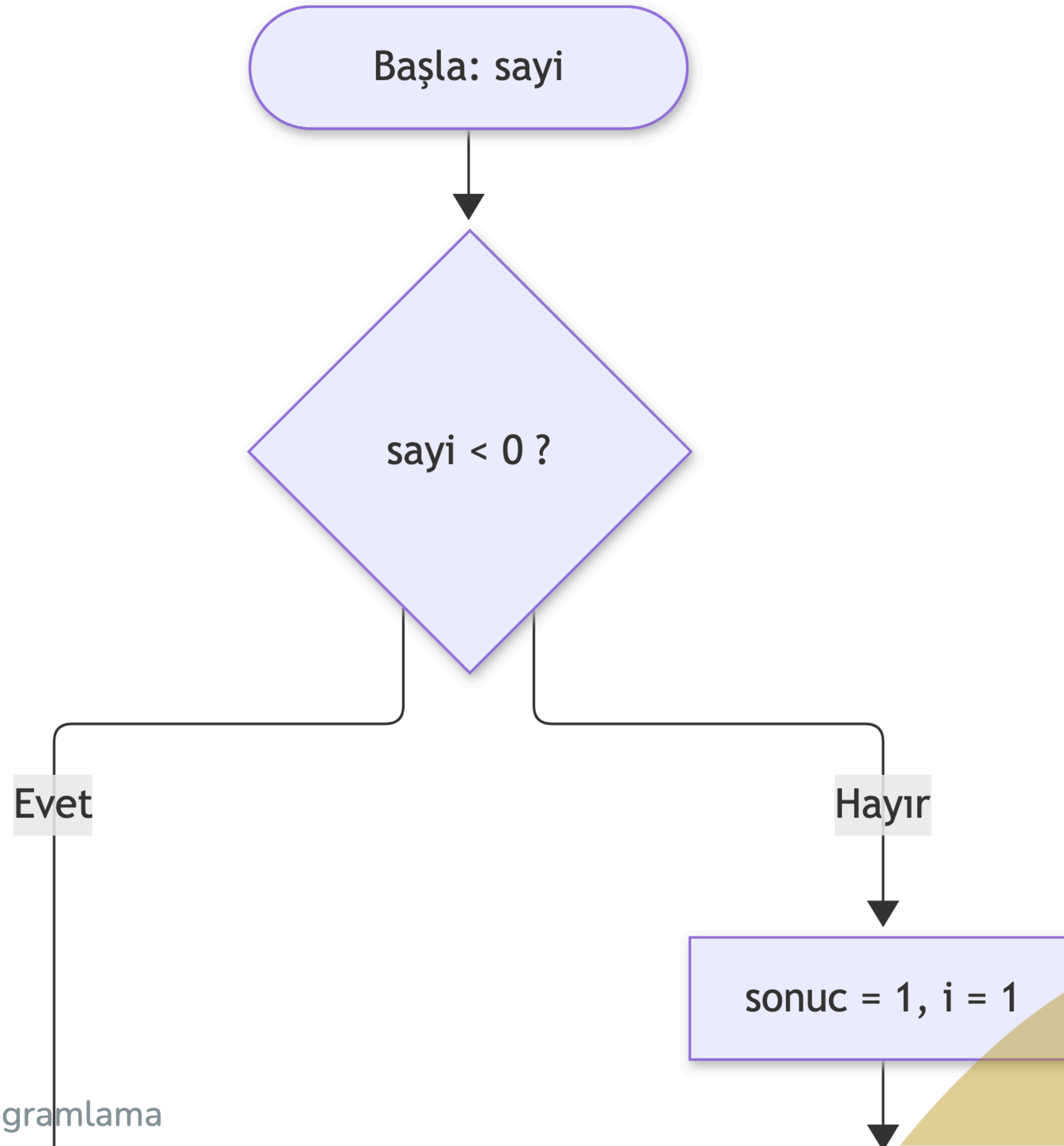
Problem:  $N!$  değerini hesaplayın ve bellek sınırlarını inceleyin.

- Matematiksel Formül:

$$N! = 1 \times 2 \times 3 \times \dots \times N \quad (0! = 1)$$

- Kritik Mühendislik Riski: Tamsayı Taşması (Integer Overflow)
- 32-bit `int` maksimum  $2^{31} - 1 \approx 2.14 \times 10^9$  tutabilir.
- $12! = 479\,001\,600$  ( `int` sığar)
- $13! = 6\,227\,020\,800$  ( `int` taşar ve negatif sonuç üretir!)
- Bu nedenle 64-bit `long long` kullanılmalıdır (en fazla  $20!$  saklar).

# Faktöriyel Akış Şeması



# Alıştırma 3: Faktöriyel C++ Çözümü

`long long` ile güvenli faktöriyel hesabı:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int sayi;
6      cout << "Pozitif bir tamsayı girin (0 - 20): ";
7      cin >> sayi;
8
9      if (sayi < 0) {
10         cout << "[HATA] Negatif sayıların faktöriyeli tanımlı değildir!" << endl;
11     } else if (sayi > 20) {
12         cout << "[UYARI] 20'den büyük sayılar 64-bit tamsayı sınırını (long long) aşar!"
13             << endl;
14     } else {
15         long long faktoriyel = 1;
16         for (int i = 1; i <= sayi; i++) {
17             faktoriyel *= i;
18         }
19         cout << sayi << "! = " << faktoriyel << endl;
20     }
```

# Alıştırma 4: Dinamik İstatistik Analizörü

Problem: Kullanıcı `0` girene kadar pozitif sayılar alarak istatistik üretin.

- İstenenler:
  1. Girilen geçerli sayı adedi
  2. En büyük sayı (*Max*)
  3. En küçük sayı (*Min*)
  4. Sayıların aritmetik ortalaması
- Tasarım Mantığı:
  - İlk girilen sayı hem *Min* hem *Max* olarak kabul edilir.
  - Sonraki sayılar geldikçe bu değerler güncellenir.
  - `0` girildiğinde döngü sonlanır ( `do-while` ideali).

# Tamsayı Bölmesi Tuzağı

Ortalama hesaplanırken:

```
1 double ortalama = toplam / sayiAdedi; // HATA!
```

- `toplam` ve `sayiAdedi` her ikisi de `int` ise C++ tamsayı bölmesi (*integer truncation*) yapar ve küsüratı atar!
- Doğru Yöntem ( `static_cast` ):

```
1 double ortalama =  
2     static_cast<double>(toplam) / sayiAdedi;
```

# Alıştırma 4: Dinamik Analizör C++ Kodu

Kullanıcı kontrollü `do-while` veri toplayıcı:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int sayi = 0, sayiAdedi = 0, toplam = 0;
6      int enBuyuk = 0, enKucuk = 0;
7
8      cout << "Pozitif sayılar girin (Durdurmak için 0 girin):" << endl;
9      do {
10         cout << "Sayı: ";
11         cin >> sayi;
12
13         if (sayi > 0) {
14             sayiAdedi++;
15             toplam += sayi;
16
17             if (sayiAdedi == 1) { // İlk geçerli sayı referans alınır
18                 enBuyuk = sayi;
19                 enKucuk = sayi;
20             } else {
```

# Alıştırma 5: Gün Belirleyici Durum Makinesi

Kullanıcıdan gün numarası (1-7) alıp gün adını belirleme:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int gun;
6      cout << "Gün numarası girin (1-7): ";
7      cin >> gun;
8
9      switch (gun) {
10         case 1: cout << "Pazartesi"; break;
11         case 2: cout << "Salı"; break;
12         case 3: cout << "Çarşamba"; break;
13         case 4: cout << "Perşembe"; break;
14         case 5: cout << "Cuma"; break;
15         case 6: cout << "Cumartesi"; break;
16         case 7: cout << "Pazar"; break;
17         default:
18             cout << "Geçersiz Gün!"; break;
19     }
```

# Tasarım Notları

- Çoklu `if-else` yerine `switch-case` yapısının temizliğine mükemmel bir örnektir.
- `default:` dalı, 1-7 dışındaki tüm hatalı girdileri (ör. 0, 9, -3) yakalar.
- Her durumun sonunda `break;` unutulmamıştır.
- Mekatronik sistemlerde bu yapı durum makinelerinde (*State Machine*) sistem modunu ( `case STANDBY:` , `case RUNNING:` ) yürütmek için kullanılır.

# Alıştırma 1: Basamak Toplama Algoritması

Açıklama: Kullanıcıdan pozitif bir tamsayı alın ve basamaklarının toplamını hesaplayın. (Örnek: `1234` için  $1 + 2 + 3 + 4 = 10$ ).

- Algoritmik İpuçları:
  - `sayi % 10` işlemi sayının en sağdaki basamağını verir.
  - `sayi / 10` işlemi tamsayı bölmesi yaparak o basamağı sayıdan atar.
  - Bu iki işlemi `sayi > 0` olduğu sürece bir `while` döngüsünde tekrarlayın.

# İskelet Kod Şablonu

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int sayi, toplam = 0;
6      cout << "Bir sayı girin: ";
7      cin >> sayi;
8
9      // TODO: while döngüsü kurun
10     // 1. Son basamağı toplama ekleyin
11     // 2. Sayıyı 10'a bölün
12
13     cout << "Basamak Toplamı: " << toplam << endl;
14     return 0;
15 }
```

# Alıştırma 2: FizzBuzz Problemi

1'den  $N$ 'e kadar olan sayıları sırayla ekrana yazdırın:

- Sayı 3'e tam bölünüyorsa sayı yerine `"Fizz"`
- Sayı 5'e tam bölünüyorsa sayı yerine `"Buzz"`
- Hem 3'e hem 5'e tam bölünüyorsa `"FizzBuzz"`
- Hiçbirine bölünmüyorsa sayının kendisini yazdırın.

# Kritik Kontrol Sıralaması

```
1  for (int i = 1; i <= n; i++) {
2      // DİKKAT: İlk önce 15'e (3 ve 5) bölünebilme
3      // kontrol edilmelidir!
4      if (i % 15 == 0) {
5          cout << "FizzBuzz ";
6      } else if (i % 3 == 0) {
7          cout << "Fizz ";
8      } else if (i % 5 == 0) {
9          cout << "Buzz ";
10     } else {
11         cout << i << " ";
12     }
13 }
```

# Mini Proje: Konsol Tabanlı Mekatronik Terminali

Öğrencilerden beklenen dönem içi mini proje çalışması:

- Proje Konusu: Bir robot veya endüstriyel test istasyonu için konsol arayüzü geliştirme.
- Zorunlu Gereksinimler:
  1. `do-while` ile sürekli aktif kalan bir ana menü döngüsü.
  2. `switch-case` ile komut yönlendirme (1: Hız Ayarla, 2: Sensörleri Listele, 3: Acil Stop, 0: Kapat).
  3. `if-else` ile sınır kontrolleri (Örn. Hız değeri 0 ile 100 arasında değilse uyarı verip reddetme).
  4. Hatalı seçimlerde `default` ile kullanıcıyı uyarma.
  5. Acil durdurma emrinde döngüyü `break` ile güvenli şekilde kapatma.

# Hafta 2 Özeti ve Kritik Kurallar

Bu haftanın en önemli kuralları ve ilkeleri:

- 1. Dallanma Tercihi: Aralıklar ve karmaşık koşullar için `if-else`, ayırık tamsayı/karakter sabitleri için `switch-case` kullanılır.
- 2. Atama Tuzağı: Koşul içinde `=` (atama) değil, mutlaka `==` (karşılaştırma) kullanılmalıdır.
- 3. Döngü Türü Seçimi:
  - Tekrar adedi belliyse → `for` (veya C++11 Range-based `for`)
  - Ön koşula bağlıysa → `while`
  - En az bir kez çalışması zorunluysa → `do-while`
- 4. Döngü Bitişleri: `do-while` haricindeki döngü ve koşul başlıklarının sonuna asla noktalı virgül ( `;` ) konulmaz!
- 5. Bellek Taşması: Faktöriyel ve seri hesaplarında veri tipi sınırlarına ( `int` vs `long long` ) dikkat edilmelidir.

# Gelecek Hafta: Hafta 3 Önizleme

Önümüzdeki hafta modüler programlamaya ve bellek mimarisine adım atıyoruz:

- Fonksiyon Tanımlama ve Bildirim: Prototip kavramı, başlık dosyaları.
- Bellek Modeli ve Parametre Aktarımı:
- Değer ile aktarım (*Pass-by-Value*) ve kopya maliyeti
- Referans ile aktarım (*Pass-by-Reference* - `&`) ve doğrudan bellek manipülasyonu
- Fonksiyon Aşırı Yükleme (Function Overloading): Aynı isimle farklı imzalı metotlar.
- Yığın Belleği (Call Stack): Fonksiyon çağrılarının stack çerçeveleri (*stack frame*) ve yerel değişken ömürleri.

Nesne Yönelimli Programlama • Marmara Üniversitesi Mekatronik Mühendisliği